

Web Application Development II

Project Documentation

Name: Menara Sathmini Perera

NIC: 200582800963

Batch: Narwhal Uni

1. Acknowledgement

I would like to express my sincere gratitude to my lecturers and supervisors for their guidance throughout this project. I am especially thankful to my classmates for their feedback and encouragement, and to my family for their continuous support. Special thanks go to everyone who provided information and ideas to help me design and develop the “Greennora” e-commerce website.

Table of Contents

No table of contents entries found.

3. Introduction

The “Greennora” e-commerce platform is an online store designed for customers to purchase plants, seeds, fertilizers, and pots from the comfort of their homes. The platform aims to simplify the buying process, provide detailed product information, and offer secure payment options. Customers can browse categories, add products to their cart, manage their profiles, and place orders.

Technologies Used:

I. Backend Java EE7 (Servlets,Hibernat)
II. Frontend HTML, CSS, JavaScript
III. Database MySQL

4. Problem Analysis

Traditional gardening product businesses rely heavily on physical stores, which limits customer reach and convenience. Many customers face difficulties such as:

- Limited store hours
- Lack of product variety in local shops
- Difficulty finding detailed product information
-

5. Project Plan

5.1. Project Proposal Content

- ### Features of the system

I. User Sign In/Sign Up
II. Home page (Catgeory with images)
III. All Product
IV. Single product view
V. Add to cart, paymenets and checkout
VI. Admin login
VII. Admin panel
VIII. Add product
IX. All product, pagination and search

- ### Limitations

I. Payments gateway is simulated (no real transaction)
II. No mobile app version

- Future enhancements

I. Mobile app (React Native)
II. Real Payments integration
III. AI- based recommendations

6. Research

Methods:

I. Questionnaries to customer to understand buying behavior
II. Interviews with store manager for business pain points
III. Competitor analysis for features and order tracking

Findings:

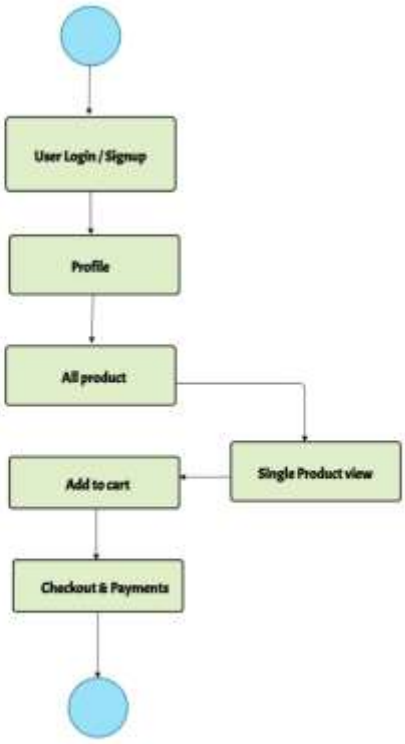
I. Customers want filtered searches and fast checkout

II. Store owners want real-time reports and order tracking

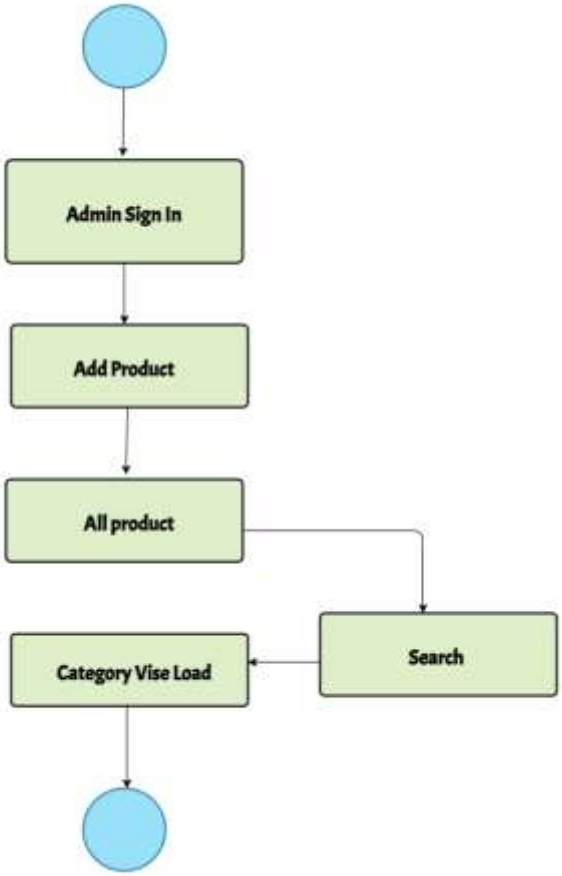
7. Analysis

7.1 Business Process Diagram

User Side



Admin Side



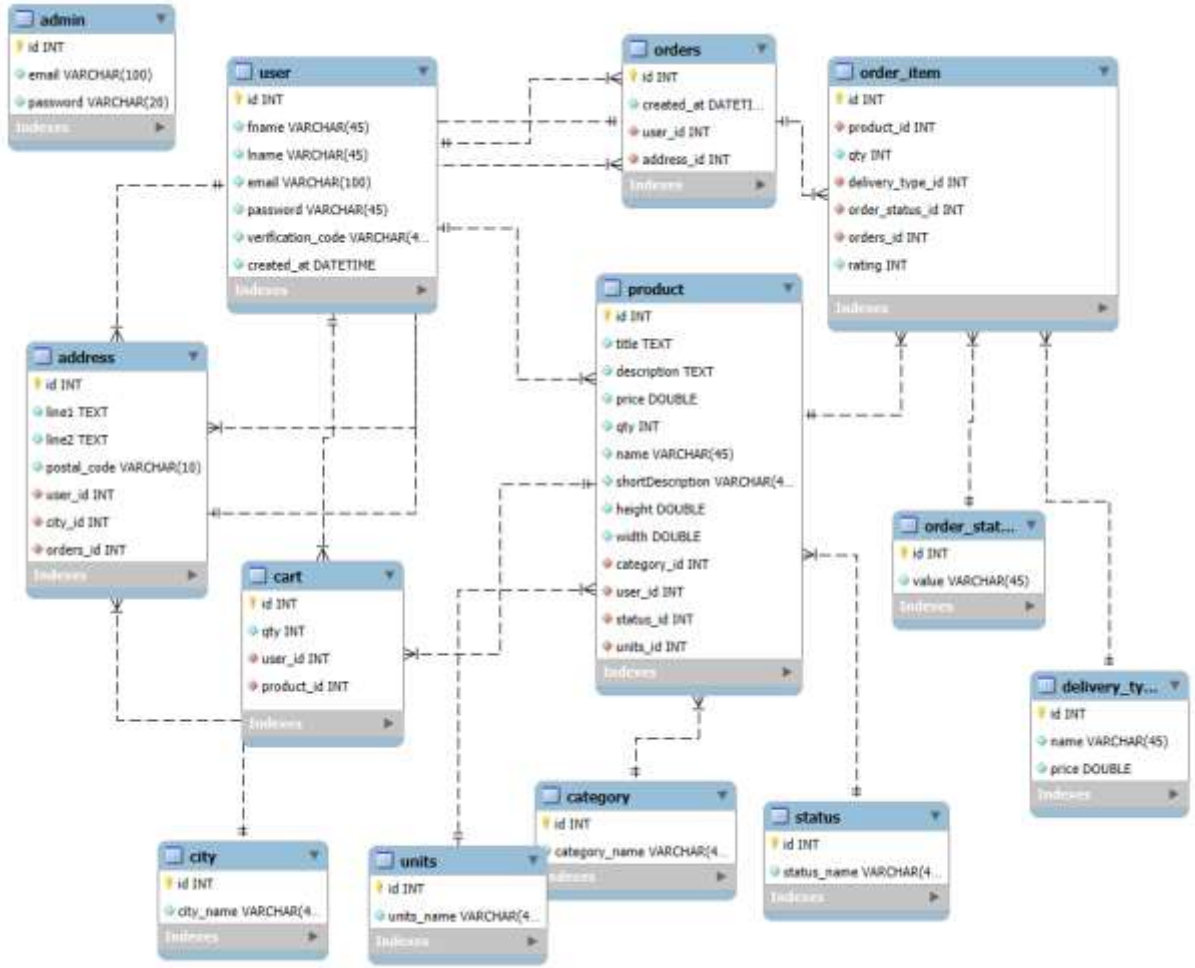
8. Design

8.1. System Design

8.1.1. Activity Diagram

8.2. Database Design

8.2.1. ER Diagram



8.2.2. Database Schema (generated via Hibernate mapping)







```
@Entity
@Table(name = "cart")
public class Cart implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "qty")
    private int qty;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    public Cart() {
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public int getQty() {
        return qty;
    }

    public void setQty(int qty) {
        this.qty = qty;
    }

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }

    public Product getProduct() {
        return product;
    }

    public void setProduct(Product product) {
        this.product = product;
```



```
@Entity
@Table(name = "category")
public class Category implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "category_name", length = 45, nullable = false)
    private String category_name;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getCategory_name() {
        return category_name;
    }

    public void setCategory_name(String category_name) {
        this.category_name = category_name;
    }
}
```

```java
@Entity
@Table(name = "city")
public class City implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "city_name", nullable = false, length = 45)
    private String city_name;

    /**
     * @return the id
     */
    public int getId() {
        return id;
    }

    /**
     * @param id the id to set
     */
    public void setId(int id) {
        this.id = id;
    }

    /**
     * @return the city_name
     */
    public String getCity_name() {
        return city_name;
    }

    /**
     * @param city_name the city_name to set
     */
    public void setCity_name(String city_name) {
        this.city_name = city_name;
    }
```





```
@Entity
@Table(name="order_item")
public class OrderItems implements Serializable{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="id")
    private int id;

    @ManyToOne
    @JoinColumn(name="product_id")
    private Product product;

    @Column(name = "qty", nullable = false)
    private int qty;

    @ManyToOne
    @JoinColumn(name="order_status_id")
    private OrderStatus orderStatus;

    @ManyToOne
    @JoinColumn(name="delivery_type_id")
    private DeliveryTypes deliveryTypes;

    @Column(name = "rating", nullable = false)
    private int rating;

    @ManyToOne
    @JoinColumn(name="orders_id")
    private Orders orders;

    public OrderItems() {
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public Product getProduct() {
        return product;
    }

    public void setProduct(Product product) {
        this.product = product;
    }

    public int getQty() {
        return qty;
    }

    public void setQty(int qty) {
        this.qty = qty;
```



```
@Entity
@Table(name="orders")
public class Orders implements Serializable{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="id")
    private int id;

    @ManyToOne
    @JoinColumn(name="user_id")
    private User user;

    @Column(name="created_at")
    private Date createdAt;

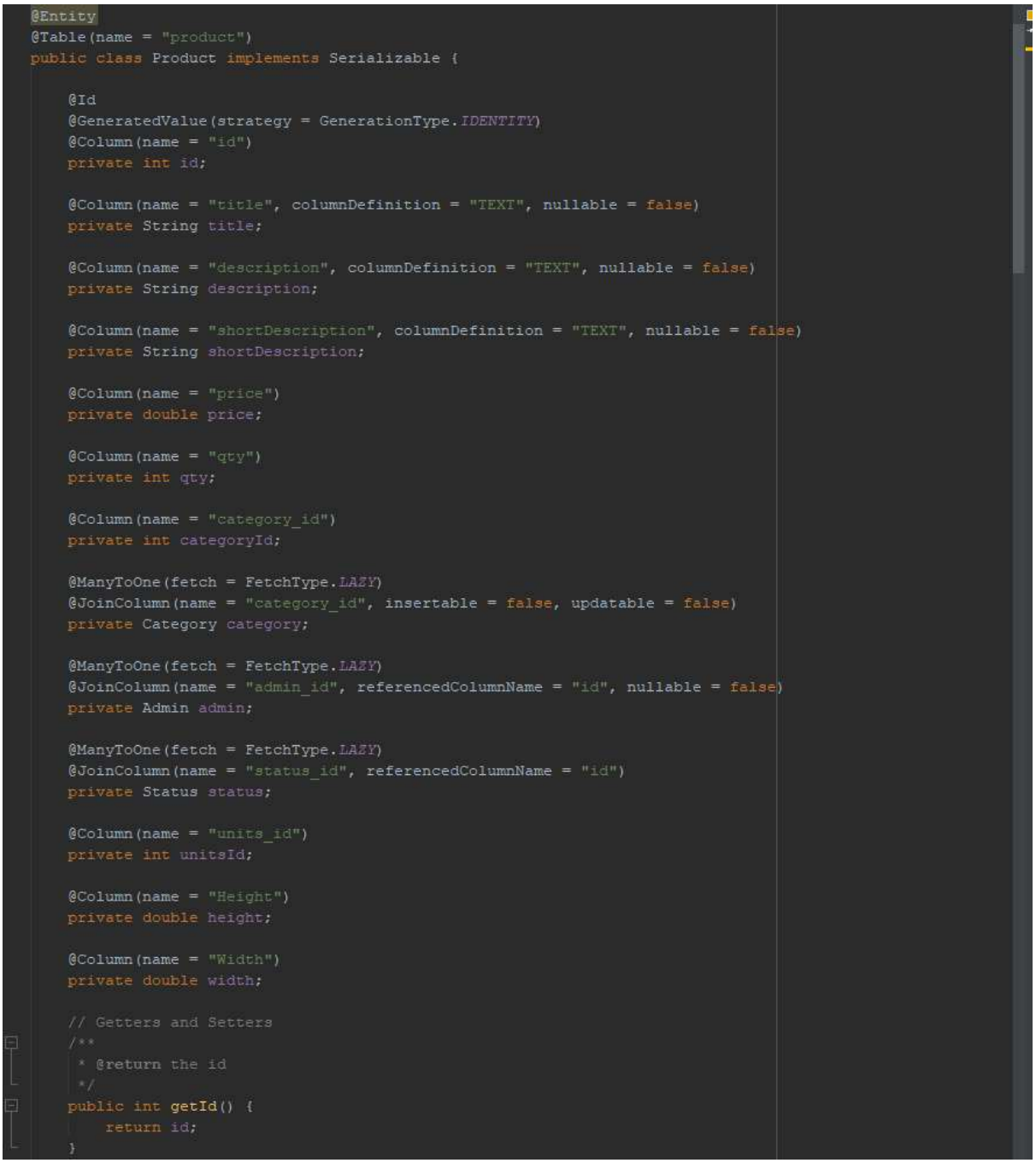
    @ManyToOne
    @JoinColumn(name="address_id")
    private Address address;

    public Orders() {
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public User getUser() {
        return user;
    }
```



```java
@Entity
@Table(name = "product")
public class Product implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "title", columnDefinition = "TEXT", nullable = false)
    private String title;

    @Column(name = "description", columnDefinition = "TEXT", nullable = false)
    private String description;

    @Column(name = "shortDescription", columnDefinition = "TEXT", nullable = false)
    private String shortDescription;

    @Column(name = "price")
    private double price;

    @Column(name = "qty")
    private int qty;

    @Column(name = "category_id")
    private int categoryId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id", insertable = false, updatable = false)
    private Category category;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "admin_id", referencedColumnName = "id", nullable = false)
    private Admin admin;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "status_id", referencedColumnName = "id")
    private Status status;

    @Column(name = "units_id")
    private int unitsId;

    @Column(name = "Height")
    private double height;

    @Column(name = "Width")
    private double width;

    // Getters and Setters
    /**
     * @return the id
     */
    public int getId() {
        return id;
    }
```



```java
@Entity
@Table(name = "status")
public class Status implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;
    @Column(name = "status_name", length = 20, nullable = false)
    private String status_name;

    public Status() {
    }

    /**
     * @return the id
     */
    public int getId() {
        return id;
    }

    /**
     * @param id the id to set
     */
    public void setId(int id) {
        this.id = id;
    }

    /**
     * @return the status_name
     */
    public String getStatus_name() {
        return status_name;
    }

    /**
     * @param status_name the status_name to set
     */
    public void setStatus_name(String status_name) {
        this.status_name = status_name;
    }
```



```java
@Entity
@Table(name = "units")
public class Units implements Serializable {

    @Id
    @Column(name = "id")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "units_name", length = 40, nullable = false)
    private String units_name;

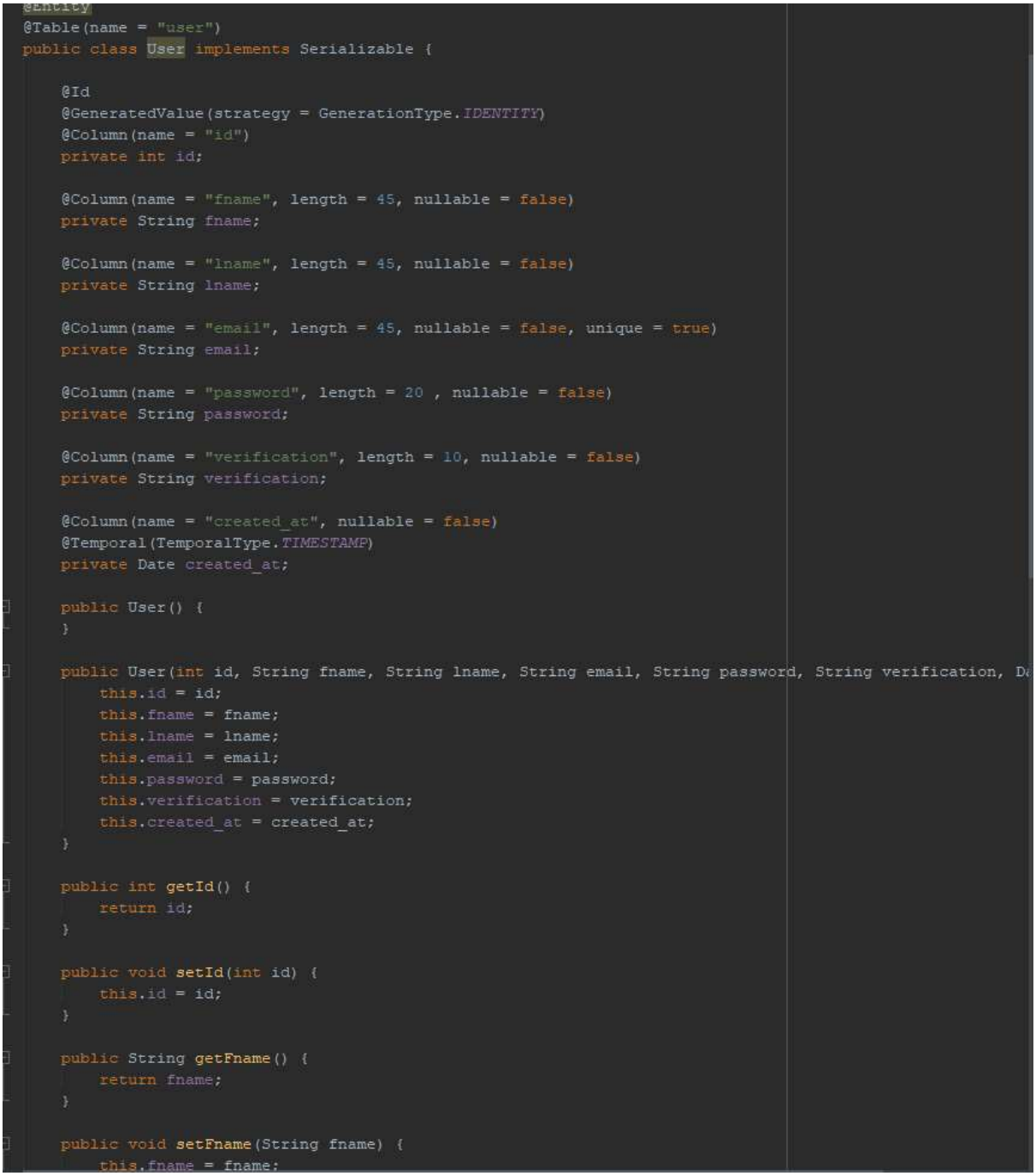
    /**
     * @return the id
     */
    public int getId() {
        return id;
    }

    /**
     * @param id the id to set
     */
    public void setId(int id) {
        this.id = id;
    }

    /**
     * @return the units_name
     */
    public String getUnits_name() {
        return units_name;
    }

    /**
     * @param units_name the units_name to set
     */
    public void setUnits_name(String units_name) {
        this.units_name = units_name;
    }


}
```



```java
@Entity
@Table(name = "user")
public class User implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "fname", length = 45, nullable = false)
    private String fname;

    @Column(name = "lname", length = 45, nullable = false)
    private String lname;

    @Column(name = "email", length = 45, nullable = false, unique = true)
    private String email;

    @Column(name = "password", length = 20 , nullable = false)
    private String password;

    @Column(name = "verification", length = 10, nullable = false)
    private String verification;

    @Column(name = "created_at", nullable = false)
    @Temporal(TemporalType.TIMESTAMP)
    private Date created_at;

    public User() {
    }

    public User(int id, String fname, String lname, String email, String password, String verification, D
        this.id = id;
        this.fname = fname;
        this.lname = lname;
        this.email = email;
        this.password = password;
        this.verification = verification;
        this.created_at = created_at;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getFname() {
        return fname;
    }

    public void setFname(String fname) {
        this.fname = fname;
```

Hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration  PUBLIC
    "-//Hibernate/Hibernate  Configuration  DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
```

```
    <session-factory>
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
        <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/greennora?useSSL=fal
se</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">Mena2005</property>
        <property name="hibernate.show_sql">true</property>

        <mapping class="hibernate.User"/>
        <mapping class="hibernate.City"/>
        <mapping class="hibernate.Address"/>
        <mapping class="hibernate.Category"/>
        <mapping class="hibernate.Status"/>
        <mapping class="hibernate.Units"/>
        <mapping class="hibernate.Product"/>
        <mapping class="hibernate.Admin"/>
        <mapping class="hibernate.Cart"/>
        <mapping class="hibernate.DeliveryTypes"/>
        <mapping class="hibernate.OrderItems"/>
        <mapping class="hibernate.Orders"/>
        <mapping class="hibernate.OrderStatus"/>

    </session-factory>
</hibernate-configuration>
```

8.3. Storyboards (Wireframes)